DESIGN AND ANALYSIS OF ALGORITHM

Finding time complexity

Question 1:

Convert the following algorithm into a program and find its time complexity using the counter method.

```
void function (int n)
{
   int i= 1;

   int s =1;

   while(s <= n)
   {
       i++;
       s += i;
    }
}
```

Note: No need of counter increment for declarations and scanf() and count variable printf() statements.

Input:

A positive Integer n

Output:

Print the value of the counter variable

For example:

Input	Result
9	12

Answer:

```
#include <stdio.h>

int count = 0;

void function(int n){
    int i = 1;
    count++;
    int s = 1;
    count++;
    while(s <= n){
        count++;
        i++;
        count++;
        s+=i;
        count++;
    }
    count++;
}

int main(){
    int n;
    scanf("%d", &n);
```

```
    function(n);
    printf("%d", count);
    return 0;
}
```

Question2:

Convert the following algorithm into a program and find its time complexity using the counter method.

```
void func(int n)
{
    if(n==1)
    {
      printf("*");
    }
    else
    {
     for(int i=1; i<=n; i++)
     {
       for(int j=1; j<=n; j++)
       {
          printf("*");
          printf("*");
          break;
       }
     }
  }
```

}

Note: No need of counter increment for declarations and scanf() and count variable printf() statements.

Input:

A positive Integer n

Output:

Print the value of the counter variable

Answer:

```
#include <stdio.h>


int count = 0;


void func(int n){
   if(n==1){
      count++;
      //printf("*");
      count++;
   }
   else{
      count++;
      for(int i = 1; i <= n; i++){
         count++;
         for(int j = 1; j <= n; j++){
            count++;
            //printf("*");
            count++;
            //printf("*");
            count++;
```

```
            break;
        }
        count++;
    }
    count++;
  }
}

int main(){
  int n;
  scanf("%d", &n);
  func(n);
  printf("%d", count);
  return 0;
}
```

Question 3:

Convert the following algorithm into a program and find its time complexity using counter method.

```
Factor(num) {
{
  for (i = 1; i <= num;++i)
  {
   if (num % i== 0)
     {
       printf("%d ", i);
     }
   }
 }
```

Note: No need of counter increment for declarations and scanf() and counter variable printf() statement.

Input:

A positive Integer n

Output:

Print the value of the counter variable

Answer:

```
#include <stdio.h>


int count = 0;


void Factor(int num){
    for(int i = 1; i <= num; i++){
        count++;
        if(num%i==0){
            //printf("%d", i);
            count++;
        }
        count++;
    }
    count++;
}


int main(){
    int n;
    scanf("%d", &n);
    Factor(n);
    printf("%d", count);
```

```
    return 0;
}
```

Convert the following algorithm into a program and find its time complexity using counter method.

```
void function(int n)
{
    int c= 0;
    for(int i=n/2; i<n; i++)
        for(int j=1; j<n; j = 2 * j)
            for(int k=1; k<n; k = k * 2)
                c++;
}
```

Note: No need of counter increment for declarations and scanf() and count variable printf() statements.

Input:

A positive Integer n

Output:

Print the value of the counter variable

Answer:

```
#include <stdio.h>


int count = 0;


void function(int n){
    int c = 0;
    count++;
    for(int i = n/2; i < n; i++){
```

```
        count++;
        for(int j = 1; j < n; j = 2 * j){
            count++;
            for(int k = 1; k < n; k = k * 2){
                count++;
                c++;
                count++;
            }
            count++;
        }
        count++;
    }
    count++;
}

int main(){
    int n;
    scanf("%d", &n);
    function(n);
    printf("%d", count);
    return 0;
}
```

Question 5:

Convert the following algorithm into a program and find its time complexity using counter method.

```
void reverse(int n)
{
  int rev = 0, remainder;
```

```
  while (n != 0)
   {
      remainder = n % 10;
      rev = rev * 10 + remainder;
      n/= 10;

   }
print(rev);
}
```

Note: No need of counter increment for declarations and scanf() and count variable printf() statements.

Input:

A positive Integer n

Output:

Print the value of the counter variable

Answer:

```
#include <stdio.h>


int count = 0;


void reverse(int n){
   int rev = 0, remainder;
   count++;
   while(n != 0){
      count++;
      remainder = n%10;
      count++;
      rev = rev * 10 + remainder;
      count++;
```

```
        n /= 10;
        count++;
    }
    count++;
    //print(rev);
    count++;
}

int main(){
    int n;
    scanf("%d", &n);
    reverse(n);
    printf("%d", count);
    return 0;
}
```